

Implementation Evidence – API-Based Multi-Model Testing

StackEval project documentation draft

1. First OpenAI API Test (Initial Step)

```
1  from openai import OpenAI
2  import os
3  import json
4  from dotenv import load_dotenv
5
6  # Load environment variables from .env
7  load_dotenv()
8
9  # Create OpenAI client
10 client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
11
12 # Test question
13 question = "Why do I get a NullPointerException in Java?"
14
15 # Send request to model
16 response = client.responses.create(
17     model="gpt-5.4",
18     input=question
19 )
20
21 # Get clean answer text
22 answer_text = response.output_text
23
24 # Build result object for JSON storage
25 result = {
26     "question": question,
27     "model": "gpt-5.4",
28     "answer": answer_text
29 }
30
31 # Save machine-readable result
32 with open("openai_test_result.json", "w", encoding="utf-8") as json_file:
33     json.dump(result, json_file, indent=4, ensure_ascii=False)
```

I started by testing the system with a single OpenAI model. The main goal of this step was to check whether the API connection worked correctly and whether the system could successfully send a programming question and receive a valid response.

2. OpenAI Result (Proof of Working API)

```

PS C:\Users\koran\Desktop\Semester 4 GenAI\First Project\test> py test_openai.py

=== MODEL OUTPUT ===

Question:
Why do I get a NullPointerException in Java?

Model:
gpt-5.4

Answer:
A `NullPointerException` in Java happens when your code tries to use `null` as if it were a real object.

## What `null` means
`null` means "no object is assigned here."

So if you do something like:

```java
String name = null;
System.out.println(name.length());
```

Java throws a `NullPointerException` because `name` does not point to a `String` object, and you're trying to call `.length()` on it.

## Common causes
You usually get this when you:

- call a method on a `null` reference
- access a field on a `null` reference
- use array access on a `null` array
- unbox a `null` wrapper type like `Integer`

### Examples

#### 1. Calling a method on `null`
```java
Person p = null;
p.getName(); // NullPointerException
```

```

The model returned a successful and detailed answer. This confirmed that the API connection was working, that the model could generate meaningful output, and that the response could be captured programmatically. This was the first clear proof that API-based querying was technically feasible for my project.

Key point: This step shows that the system did not only connect successfully, but also produced usable output that could later be stored and compared.

3. Expanding to All Models

After the first successful OpenAI test, I expanded the setup so that it could support multiple models. The selected models represent different categories in the LLM ecosystem, including a frontier model, a competitor model, an open-source model, and a budget or lightweight model. This follows the logic of the StackEval approach, where different model types are compared under the same task conditions.

4. Running All Models in One Script

All selected models were then executed within one script. The same question was sent to each model, and the responses were collected automatically. This is an important step because it creates a fairer

comparison: each model receives the same input and is tested through the same process.

```
14 openrouter_api_key = os.getenv("OPENROUTER_API_KEY")
15
16 # One question for all models
17 question = "Write a function in Python that removes duplicates from a list."
18
19 # Model list
20 models = [
21     {
22         "label": "Frontier proprietary model",
23         "provider": "openai",
24         "model_name": "gpt-5.4"
25     },
26     {
27         "label": "Frontier competitor",
28         "provider": "anthropic",
29         "model_name": "claude-sonnet-4-6"
30     },
31     {
32         "label": "Open-source model",
33         "provider": "openrouter",
34         "model_name": "meta-llama/llama-4-maverick"
35     },
36     {
37         "label": "Budget / lightweight model",
38         "provider": "openai",
39         "model_name": "gpt-5.4-mini"
40     }
41 ]
42
43 results = []
44
45 for item in models:
46     label = item["label"]
```

Key point: Using one script for all models improves consistency and makes the testing setup easier to repeat later.

5. Results from All Models

```

1  default:
2    custom_llm_provider: openai
3    sample_parameters:
4      temperature: 0.01
5      max_tokens: 2048
6      seed: 42
7    model_parameters:
8      model: gpt-4-turbo-2024-04-09
9      rate_limit: 100
10   model_metadata:
11     name: GPT-4 Turbo
12     provider: OpenAI
13
14   default-json:
15     custom_llm_provider: openai
16     sample_parameters:
17       temperature: 0.01
18       max_tokens: 2048
19       seed: 42
20     response_format:
21       type: json_object
22     model_parameters:
23       model: gpt-4o-mini-2024-07-18
24       rate_limit: 200
25     model_metadata:
26       name: GPT-4o Mini
27     provider: OpenAI

```

The collected responses were saved in a structured JSON format. Each entry contains the provider, the model, the question, and the answer. This is important because it turns the outputs into a usable dataset that can later be evaluated by humans in a StackEval-style scoring process.

6. Comparison with the StackEval Approach

Image to insert: Screenshot of the StackEval-style configuration file or script settings.

The StackEval GitHub project uses structured settings such as token limits, temperature values, and model parameters. My own implementation also contains similar ideas, such as model selection, token-related settings, and structured output collection. Because of this, my setup already resembles the general structure of StackEval. However, it is still only a test implementation at this stage, while StackEval is a much more advanced and fully developed benchmark system.

```

[
  {
    "label": "Frontier proprietary model",
    "provider": "openai",
    "model": "gpt-5.4",
    "question": "Write a function in Python that removes duplicates from a list.",
    "answer": "```python\ndef remove_duplicates(items):\n    return list(dict.fromkeys(items))\n```\n\nExample:\n```python\nnums = [1, 2, 2, 3, 1, 4]\nremove_duplicates(nums)\n# Output: [1, 2, 3, 4]\n```"
  },
  {
    "label": "Frontier competitor",
    "provider": "anthropic",
    "model": "claude-sonnet-4-6",
    "question": "Write a function in Python that removes duplicates from a list.",
    "answer": "## Remove Duplicates from a List\n\nHere are several approaches to remove duplicates from a list in Python:\n\n### 1. Using a Set (O(n) time complexity)\n\n```python\ndef remove_duplicates(items):\n    return list(set(items))\n\n# Example\nitems = [1, 2, 2, 3, 1, 4]\nremove_duplicates(items)\n# Output: [1, 2, 3, 4]\n```"
  },
  {
    "label": "Open-source model",
    "provider": "openrouter",
    "model": "meta-llama/llama-4-maverick",
    "question": "Write a function in Python that removes duplicates from a list.",
    "answer": "### Removing Duplicates from a List in Python\n\n#### Function Implementation\n\n```python\ndef remove_duplicates(input_list):\n    seen = set()\n    result = []\n    for item in input_list:\n        if item not in seen:\n            result.append(item)\n            seen.add(item)\n    return result\n\n# Example\nitems = [1, 2, 2, 3, 1, 4]\nremove_duplicates(items)\n# Output: [1, 2, 3, 4]\n```"
  },
  {
    "label": "Budget / lightweight model",
    "provider": "openai",
    "model": "gpt-5.4-mini",
    "question": "Write a function in Python that removes duplicates from a list.",
    "answer": "Here's a simple Python function that removes duplicates from a list while preserving the original order:\n\n```python\ndef remove_duplicates(items):\n    seen = set()\n    result = []\n    for item in items:\n        if item not in seen:\n            result.append(item)\n            seen.add(item)\n    return result\n\n# Example\nitems = [1, 2, 2, 3, 1, 4]\nremove_duplicates(items)\n# Output: [1, 2, 3, 4]\n```"
  }
]

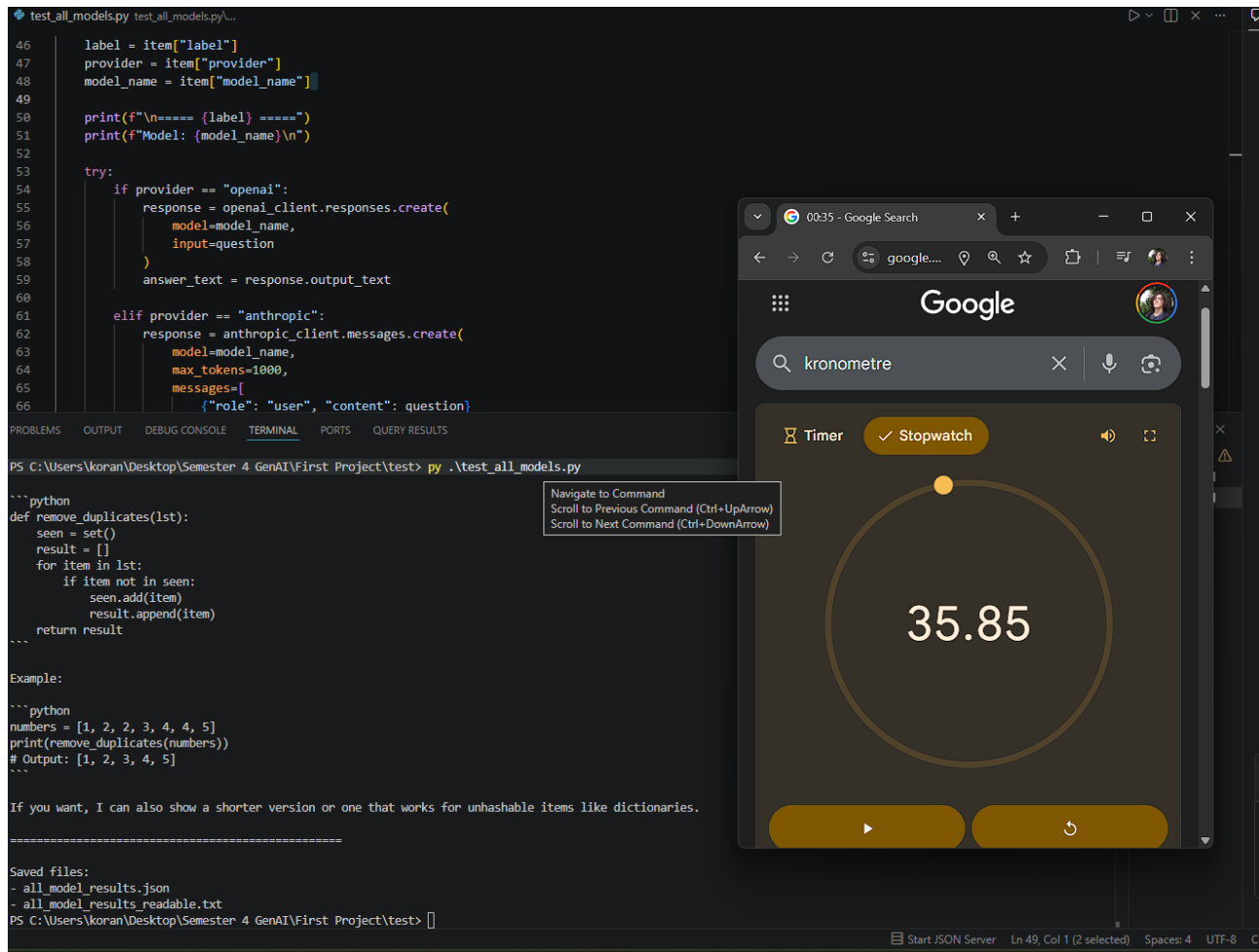
```

Key point: The important point here is not that my implementation is already equal to StackEval, but that it is clearly moving in the same methodological direction.

7. Performance Observation

Image to insert: Stopwatch or timer screenshot showing the observed runtime.

After several test runs, I observed that collecting responses from all models and saving all outputs takes approximately 30 to 35 seconds for one question. This gives a first practical indication of how long the full benchmark process may take when it is scaled to a larger question set.



Final Insight

With this implementation, API-based querying is working, multiple models can be tested under the same conditions, and the responses are automatically collected and saved. This means the system already follows the basic logic of the StackEval methodology. At the same time, this version should still be understood as a baseline test setup rather than a fully developed benchmark pipeline.